

Software Requirements Specification

Sudoku Game (C++ Console/Desktop Application)

Version 1.0

Prepared as an academic project applying C++, OOP, Data Structures, and Algorithms

Date: September 2026

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for a Sudoku puzzle game implemented in C++. The purpose of this document is to define, in a clear and structured way, what the system must do before implementation begins, so that the design (classes, data structures, and algorithms) can be planned deliberately rather than improvised while coding.

This project is intended as a practice project that consolidates concepts learned in C++, Object-Oriented Programming (OOP), Data Structures, and Algorithms into a single, complete application.

1.2 Scope

The system is a single-player Sudoku game that allows a user to:

- Play Sudoku on a standard 9x9 grid divided into nine 3x3 sub-grids (boxes).
- Choose a difficulty level (Easy, Medium, Hard) that controls how many cells are pre-filled.
- Enter, change, and clear values in empty cells.
- Receive validation feedback (row/column/box conflicts) as they play.
- Request hints and use undo/redo while solving.
- Have the puzzle auto-generated and auto-solved by the program (using a backtracking algorithm).
- Track time spent and number of moves, and save/load a game in progress.

The scope does not include multiplayer play, networked features, or a graphical 3D interface. The primary interface is a console (text-based) UI, with an optional simple GUI (e.g., using a library such as SFML or Qt) treated as an optional extension, not a core requirement.

1.3 Intended Audience

- The student/developer implementing the project.
- An instructor or reviewer evaluating the project against the OOP/DSA/Algorithms curriculum.
- Future maintainers who may extend the project (e.g., add a GUI or multiplayer).

1.4 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
Grid / Board	The 9x9 matrix of cells that makes up a Sudoku puzzle
Cell	A single unit of the grid holding a value from 1-9 or empty
Box	One of the nine 3x3 sub-grids within the 9x9 board

Backtracking	A recursive trial-and-error algorithm used to generate/solve puzzles
OOP	Object-Oriented Programming
DSA	Data Structures and Algorithms

1.5 References

- C++ Standard Library documentation (cppreference.com)
- Course material: C++, Object-Oriented Programming, Data Structures, Algorithms

2. Overall Description

2.1 Product Perspective

The Sudoku game is a standalone desktop/console application written in C++. It is a new, self-contained product (not a component of a larger system). It will be structured using OOP principles so that the puzzle logic, the solving/generation algorithms, and the user interface are separated into distinct, testable classes.

2.2 Product Functions (Summary)

- Generate a valid, solvable Sudoku puzzle with a unique solution.
- Display the board to the user and accept moves.
- Validate moves against Sudoku rules in real time.
- Provide hints, undo/redo, and a solve-for-me option.
- Detect and announce puzzle completion / win condition.
- Persist (save/load) game state to/from a file.
- Track and display statistics: elapsed time, number of hints used, number of moves, mistakes.

2.3 User Classes and Characteristics

User Type	Description
Player	Casual or student user who wants to play, pause, save, and resume Sudoku puzzles.
Developer/Evaluator	Reviews the source code structure (OOP design, data structures, algorithms) for correctness and quality.

2.4 Operating Environment

- Language: C++ (C++17 or later recommended).
- Platform: Cross-platform console application (Windows/Linux/macOS); compiled with g++/clang++/MSVC.
- No external database is required; persistence uses local text/JSON files.
- Optional GUI extension may depend on a graphics library (e.g., SFML, Qt) — out of the core scope.

2.5 Design and Implementation Constraints

- Must be implemented using Object-Oriented Programming: encapsulation, abstraction, at least one inheritance/polymorphism relationship, and clear class responsibilities.
- Must make deliberate use of data structures beyond a plain 2D array where appropriate (e.g., stacks for undo/redo, sets for candidate tracking, vectors of structs for move history).
- Puzzle generation and solving must use an explicit algorithm (backtracking with constraint checking), not a hard-coded puzzle list.

- Code must compile without warnings on a standard C++17 compiler and be organized into multiple files (.h/.hpp and .cpp) rather than a single file.

2.6 Assumptions and Dependencies

- The user has a keyboard and a terminal capable of displaying basic text/ASCII grids.
- Only the classic 9x9 Sudoku variant is required (no 4x4, 16x16, or irregular variants).
- The user has read/write permission in the folder used for save files.

3. Functional Requirements

Each requirement is uniquely identified for traceability (e.g., during testing or grading).

3.1 Puzzle Generation

ID	Requirement	Priority
FR-1	The system shall generate a complete, valid, randomly-filled 9x9 Sudoku solution using a backtracking algorithm.	High
FR-2	The system shall remove a difficulty-dependent number of cells from the completed solution to create the playable puzzle, while guaranteeing the puzzle has a unique solution.	High
FR-3	The system shall support at least three difficulty levels (Easy, Medium, Hard), each mapped to a defined range of pre-filled/removed cells.	High
FR-4	The system shall allow the player to request a brand-new puzzle at the chosen difficulty at any time.	Medium

3.2 Gameplay and Board Interaction

ID	Requirement	Priority
FR-5	The system shall display the 9x9 board with clear separation of the nine 3x3 boxes.	High
FR-6	The system shall allow the player to select a cell (by row and column) and enter a value from 1-9.	High
FR-7	The system shall prevent the player from modifying cells that were pre-filled as part of the original puzzle.	High
FR-8	The system shall allow the player to clear (erase) a value they previously entered.	Medium
FR-9	The system shall validate, after every entry, whether the value conflicts with Sudoku rules (same value in the same row, column, or 3x3 box) and shall visibly flag such conflicts.	High
FR-10	The system shall detect when the board is completely and correctly filled and shall declare the puzzle solved.	High

3.3 Player Assistance

ID	Requirement	Priority
FR-11	The system shall provide a 'hint' function that reveals the correct value for one empty cell of the player's choosing (or an automatically chosen empty cell), and shall track how many hints were used.	Medium
FR-12	The system shall provide an 'undo' function that reverts the last player move, supporting multiple consecutive undos.	Medium

FR-13	The system shall provide a 'redo' function that re-applies a move that was undone, as long as no new move has been made since.	Low
FR-14	The system shall provide a 'solve' function that automatically fills the entire remaining board using the solving algorithm.	Medium

3.4 Persistence and Statistics

ID	Requirement	Priority
FR-15	The system shall allow the player to save the current game state (board, original puzzle, elapsed time, move history) to a file.	Medium
FR-16	The system shall allow the player to load a previously saved game and resume play from that exact state.	Medium
FR-17	The system shall track and display elapsed time since the puzzle started (excluding paused time, if pause is supported).	Low
FR-18	The system shall track and display the number of moves made and the number of rule-violation mistakes.	Low

3.5 Menu and Navigation

ID	Requirement	Priority
FR-19	The system shall present a main menu allowing the player to: start a new game, load a saved game, view instructions, or exit.	High
FR-20	The system shall allow the player to return to the main menu from an active game without crashing or losing unsaved progress unexpectedly (a confirmation prompt shall be shown).	Medium

4. Non-Functional Requirements

4.1 Performance

- Puzzle generation (including uniqueness verification) shall complete in under 2 seconds on a typical development machine.
- Move validation shall be effectively instantaneous (perceived as immediate by the user, i.e., under 100 ms).

4.2 Usability

- Board and menus shall be readable in a standard 80x24 (or larger) terminal window.
- Error messages (e.g., invalid input, occupied fixed cell) shall be clear and shall not crash the program.

4.3 Reliability

- The system shall never allow an unsolvable or multiple-solution puzzle to be presented to the player.
- The system shall handle invalid user input (non-numeric, out-of-range) gracefully, without crashing.

4.4 Maintainability

- Code shall follow consistent naming conventions and be organized by responsibility (e.g., Board, Cell, SudokuGenerator, SudokuSolver, GameController, FileManager, UI).
- Each class shall have a single, well-defined responsibility to support future extension (e.g., adding a GUI).

4.5 Portability

- The codebase shall avoid platform-specific APIs where possible, or shall isolate them behind a single interface, so it can be compiled on Windows, Linux, and macOS.

5. System Design Overview (OOP, Data Structures & Algorithms)

This section is not a full design document, but gives a suggested class breakdown to guide the transition from requirements to implementation, applying the OOP and DSA concepts this project is meant to practice.

5.1 Suggested Class Structure

Class	Responsibility	Key Data Structure(s)
Cell	Represents a single board cell: its value, whether it is fixed (given), and its set of candidate values.	int value; bool isFixed; std::set<int> candidates;
Board	Owns the 9x9 grid of Cell objects; exposes row/column/box access and rule-checking.	std::array<std::array<Cell,9>,9> grid;
SudokuGenerator	Builds a complete valid solution and removes cells to create a puzzle of the chosen difficulty.	Uses backtracking; std::vector<std::pair<int,int>> for removable positions
SudokuSolver	Solves a given board (used for uniqueness-checking and the 'solve' feature).	Recursive backtracking algorithm
MoveHistory	Records moves for undo/redo.	Two std::stack<Move> (undo stack, redo stack)
GameController	Coordinates Board, Generator, Solver, MoveHistory, and the UI; enforces game rules (e.g., can't edit fixed cells).	Aggregates the above objects
FileManager	Serializes/deserializes game state to/from disk.	File I/O; simple text or JSON format
ConsoleUI (or GUI)	Renders the board and menus, and reads player input.	N/A (presentation layer)

5.2 Key Algorithms

5.2.1 Puzzle Generation and Solving — Backtracking

A recursive backtracking algorithm fills empty cells one at a time: it tries values 1-9 in the current empty cell, checks row/column/box validity, recurses into the next empty cell, and backtracks (undoes the value and tries the next candidate) whenever no value leads to a solution. The same algorithm, applied to a puzzle with several cells already filled, is used both to generate the full solution and to solve/verify puzzles.

5.2.2 Uniqueness Check

After removing cells to create a puzzle, the solver is run to confirm exactly one solution exists (e.g., by counting solutions up to 2 and stopping early), before presenting the puzzle to the player.

5.2.3 Undo/Redo

Each player move is pushed onto an undo stack as a small Move record (row, column, previous value, new value). Undo pops from the undo stack, reverts the cell, and pushes the move onto a redo stack; redo does the reverse. Making a new move after an undo clears the redo stack.

5.3 Class Relationships

- Composition: Board is composed of 81 Cell objects (Board "has-a" Cell).
- Composition: GameController is composed of a Board, a SudokuGenerator, a SudokuSolver, and a MoveHistory.
- Optional inheritance/polymorphism: a common interface (e.g., IRenderer with a virtual render(Board&) method) can be implemented by ConsoleUI and, later, a GUI class, to demonstrate polymorphism without redesigning the game logic.

6. Use Cases

6.1 Use Case: Start New Game

Field	Description
Actor	Player
Precondition	Application is running and the main menu is shown
Main Flow	1. Player selects "New Game". 2. System asks for difficulty. 3. System generates a unique puzzle. 4. System displays the board.
Postcondition	A new puzzle is ready for play

6.2 Use Case: Make a Move

Field	Description
Actor	Player
Precondition	A puzzle is in progress
Main Flow	1. Player selects a cell. 2. Player enters a value. 3. System checks the cell is not fixed. 4. System validates the move and updates the board. 5. System checks for a win condition.
Alternate Flow	If the cell is fixed, or the value conflicts with Sudoku rules, the system rejects the move and shows a message.
Postcondition	Board is updated (or unchanged, with feedback shown)

6.3 Use Case: Save / Load Game

Field	Description
Actor	Player
Precondition	A puzzle is in progress (Save) / a save file exists (Load)
Main Flow	Save: system writes board state, fixed-cell map, and elapsed time to a file. Load: system reads the file and restores the exact game state.
Postcondition	Game state is persisted or restored

7. Future Enhancements (Out of Current Scope)

- Graphical user interface (SFML, Qt, or similar).
- Multiple board sizes (4x4, 16x16) or irregular ('jigsaw') Sudoku variants.
- Online leaderboard or multiplayer race mode.
- Pencil-mark / candidate-notes mode for advanced solving strategies.

8. Approval

This document defines the baseline requirements for the Sudoku C++ project and will guide the class design, data structure choices, and algorithm implementation described in Section 5.